

Asignatura	Datos del alumno	Fecha
Métodos Numéricos	Apellidos: De Mendoza Tovar	25-05-2025
	Nombre: Alejandro	

**ACTIVIDAD LABORATORIO NO.1
ALGORITMOS, DIAGRAMAS DE FLUJO Y CÓDIGO FUENTE**

PRESENTADO POR:
ALEJANDRO DE MENDOZA

PRESENTADO AL PROFESOR:
ING HERNAN ALONSO MANCIPE BOHORQUEZ
PROFESOR

FUNDACIÓN UNIVERSITARIA INTERNACIONAL DE LA RIOJA
FACULTAD DE INGENIERÍA – INGENIERIA INFORMÁTICA
BOGOTÁ D.C.
25 DE MAYO
2025

Asignatura	Datos del alumno	Fecha
Métodos Numéricos	Apellidos: De Mendoza Tovar	25-05-2025
	Nombre: Alejandro	

Tabla de Contenido

RESUMEN	3
INTRODUCCIÓN	3
DESARROLLO DEL LABORATORIO	4
1. Encontrar las raíces reales de la ecuación $ax^2+bx+c=0$ (números decimales y método tradicional)	4
Verificación con DESMOS	15
Verificación con Google Colab	17
2. Buscar una de las raíces de la ecuación de segundo grado usando el método de la Secante	19
i. Función Principal	21
Paso 1: Declaración e Inicialización de Variables	21
Paso 2: Entrada de Datos	22
Paso 3: Cálculo de las Raíces	23
Paso 4: Salida de Resultados	23
Paso 5: Fin del Programa	23
ii. Función evaluarFx	23
Parámetros	24
Variables Locales	24
Proceso Detallado	24
Cálculo de $f(x)$	24
Asignación del resultado	24
Retorno	24
Importancia Matemática	24
iii. Función secante	24
Parámetros	25
Variables Locales	25
Proceso Detallado	25
Inicialización	25
Bucle Principal (Mientras iteracion < maxIter AND NOT terminate)	25
Análisis Matemático del Método de la Secante	27
Aplicación del Método de la Secante con los Valores Dados	27
Comportamiento Esperado	27
Conclusión	27
Ingreso de valores y pruebas	27
Verificación con DESMOS	28
Verificación con Google Colab	29
3. Diagrama de flujo con su respectiva sustentación matemática y código fuente generado para el programa que encuentra las raíces reales de la ecuación $ax^2+bx+c=0$, junto con una captura de pantalla con la ejecución en el IDE elegido	31
Flujo, código en Python y salida de consola en Flowgorithm	31
Código y salida de consola en Google Colab	31
Código, salida y grafica de consola en DESMOS	31
4. Diagrama de flujo con su respectiva sustentación matemática y código fuente generado para la técnica de la secante que busca una de las raíces de la ecuación de segundo grado, junto con una captura de pantalla con la ejecución en el IDE elegido	32
Flujo, código en Python y salida de consola en Flowgorithm	32
Función Principal	32
Función evaluarFx	32
Función Secante	33
Código y salida de consola en Google Colab (tener presente que las iteraciones se muestran en consola)	33
Código, salida y grafica de consola en DESMOS	34
CONCLUSIÓN	34
BIBLIOGRAFÍA	35

Asignatura	Datos del alumno	Fecha
Métodos Numéricos	Apellidos: De Mendoza Tovar	25-05-2025
	Nombre: Alejandro	

RESUMEN

Este laboratorio me permitió fortalecer mis competencias en el uso de algoritmos, diagramas de flujo, pruebas de escritorio y programación, especialmente enfocados en la implementación de métodos numéricos para la resolución de ecuaciones cuadráticas. Para ello, trabajé con las herramientas Flowgorithm, Google Colab y Desmos, que me facilitaron el análisis desde una perspectiva tanto visual como computacional.

Inicié resolviendo la ecuación cuadrática $ax^2 + bx + c = 0$ utilizando el método tradicional, aplicando la fórmula general. Esta solución exacta me sirvió como punto de referencia para comparar posteriormente los resultados obtenidos con el método numérico.

Después, desarrollé en Flowgorithm el algoritmo correspondiente al método de la secante, lo cual me permitió estructurar la lógica del proceso iterativo de forma clara y visual. Esta técnica resulta muy útil cuando se requiere aproximar raíces reales en situaciones donde no es posible o no conviene aplicar soluciones analíticas.

Posteriormente, exporté el pseudocódigo a Python y lo ejecuté en Google Colab, utilizando los mismos valores de entrada que en el paso anterior. Durante esta fase ajusté parámetros como la tolerancia (ϵ) y el número máximo de iteraciones, para garantizar la precisión y estabilidad del método. Verifiqué los resultados y corregí detalles del código hasta asegurar que el algoritmo se comportara como se esperaba.

Finalmente, usé Desmos para graficar la función cuadrática, junto con las raíces obtenidas por ambos métodos. Esta visualización fue clave para validar que el método de la secante efectivamente converge hacia una de las raíces, y para comparar su precisión frente al resultado del método tradicional.

Este laboratorio no solo me permitió resolver una ecuación mediante diferentes métodos, sino también integrar herramientas y enfoques complementarios. Destacó la importancia de los diagramas de flujo para organizar ideas, las pruebas de escritorio para depurar errores, y la programación como medio para automatizar y validar resultados matemáticos de forma precisa.

INTRODUCCIÓN

En el marco del estudio de algoritmos y métodos numéricos, el presente laboratorio tiene como propósito aplicar y reforzar conocimientos relacionados con la resolución de ecuaciones cuadráticas, integrando el uso de diagramas de flujo, pseudocódigo y programación. Para ello, utilicé la herramienta Flowgorithm como medio para diseñar y visualizar la lógica de los algoritmos, y posteriormente traduje dichos algoritmos al lenguaje Python, ejecutándolos en Google Colab. Además, empleé Desmos para representar gráficamente las funciones cuadráticas y validar visualmente los resultados obtenidos.

El enfoque principal se centró en resolver la ecuación cuadrática $ax^2 + bx + c = 0$, primero mediante el método tradicional (fórmula general) y luego a través del método de la secante, una técnica iterativa que permite aproximar raíces reales sin necesidad de calcular derivadas. Esta combinación de enfoques analíticos y numéricos me permitió comparar resultados, analizar el comportamiento del algoritmo, y reflexionar sobre la utilidad de cada método en diferentes contextos.

La actividad también busca resaltar cómo estas herramientas y técnicas pueden aplicarse a situaciones reales, como la modelación de fenómenos en ciberseguridad o el análisis de comportamiento no lineal en sistemas informáticos. A través del desarrollo de esta práctica, se pretende no solo resolver un problema matemático, sino también comprender y aplicar un

Asignatura	Datos del alumno	Fecha
Métodos Numéricos	Apellidos: De Mendoza Tovar	25-05-2025
	Nombre: Alejandro	

enfoque sistemático para el diseño, implementación y validación de soluciones computacionales.

DESARROLLO DEL LABORATORIO

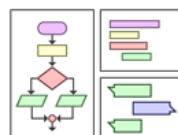
Para iniciar este laboratorio en ciberseguridad y la gestión de sistemas, el crecimiento de amenazas o la carga del sistema pueden seguir patrones no lineales, es así como las ecuaciones cuadráticas pueden servir para modelar estos crecimientos. Sus **raíces** representan los puntos críticos donde el sistema alcanza umbrales importantes en términos de carga, desempeño o vulnerabilidad, estos puntos son eventos críticos en los que un sistema podría colapsar, perder estabilidad o entrar en un estado de riesgo frente a alguna amenaza, lo que podría ser clave para saber cuándo un sistema necesita atención urgente para evitar fallos o brechas de seguridad.

A continuación, procedo con el respectivo desarrollo de cada uno de los puntos:

1. Encontrar las raíces reales de la ecuación $ax^2+bx+c=0$ (números decimales y método tradicional)

Ahora para iniciar este laboratorio y el desarrollo de este primer punto, lo primero es encontrar las raíces reales de la ecuación $ax^2 + bx + c = 0$. Donde debo elegir los coeficientes a y b con una parte decimal, por ejemplo: $a = 4.5, b = 10.7$:

Entonces lo primero es descargar Flowgorithm y luego que lo descargo selecciono la opción de Disposición de ventanas y elijo Código y Consola



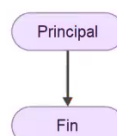
Código y consola

Y de esta manera se me muestran 3 ventanas:

- Lado izquierdo: Es donde voy a realizar el diagrama de flujo
- Lado derecho arriba: Donde se genera el código fuente
- Lado derecho abajo: Donde se incorporan los coeficientes o variables de entrada.

Método Tradicional:

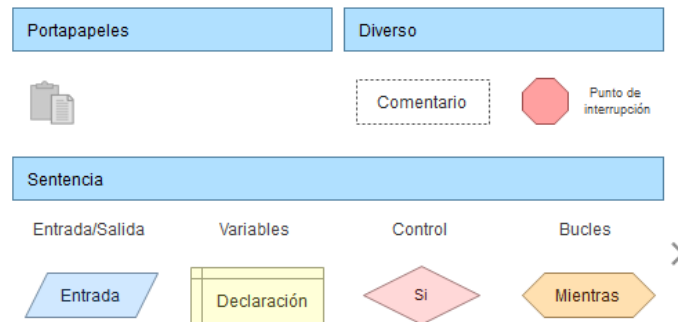
Entonces lo primero es diseñar el flujo, entonces voy a la ventana del lado izquierdo y se me muestra:



Y eso lo que indica es el principio y el fin del diagrama de flujo.

Entonces lo siguiente es dar clic en la flecha que une el principio el fin e indico y nos muestra la siguiente ventana:

Asignatura	Datos del alumno	Fecha
Métodos Numéricos	Apellidos: De Mendoza Tovar	25-05-2025
	Nombre: Alejandro	



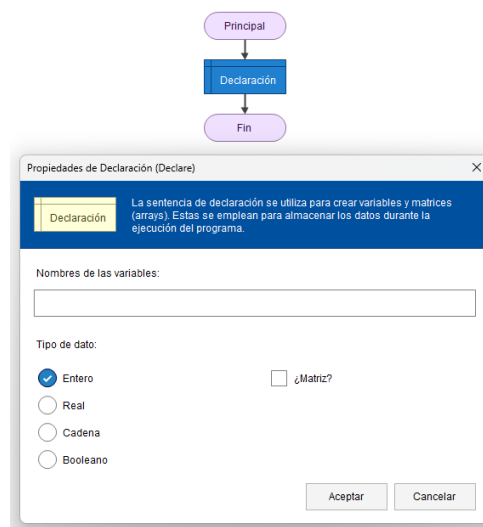
Ahora voy a generar una asignación de un componente en el campo de sentencia, y en este caso lo primero es declarar una variable, entonces doy clic en Declaración:



Y al dar clic entonces se nos genera la siguiente imagen en el flujo:



Ahora doy doble clic en declaración y se nos muestra la siguiente ventana:



Y acá incorporo mis coeficientes que en este caso son a, b y c, entonces donde se indica nombres de las variables, para declarar el primer coeficiente coloco la letra a que es mi primer coeficiente:

Asignatura	Datos del alumno	Fecha
Métodos Numéricos	Apellidos: De Mendoza Tovar	25-05-2025
	Nombre: Alejandro	

Propiedades de Declaración (Declare)

Declaración

La sentencia de declaración se utiliza para crear variables y matrices (arrays). Estas se emplean para almacenar los datos durante la ejecución del programa.

Nombres de las variables:

a

Tipo de dato:

☒ Entero ☐ ¿Matriz?

☐ Real

☐ Cadena

☐ Booleano

Aceptar Cancelar

Ahora procedo a indicar el tipo de dato y como en este caso voy a trabajar con números decimales entonces coloco que el tipo de dato es un dato real y la imagen queda de la siguiente manera:

Propiedades de Declaración (Declare)

Declaración

La sentencia de declaración se utiliza para crear variables y matrices (arrays). Estas se emplean para almacenar los datos durante la ejecución del programa.

Nombres de las variables:

a

Tipo de dato:

☐ Entero ☐ ¿Matriz?

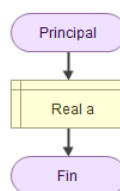
☒ Real

☐ Cadena

☐ Booleano

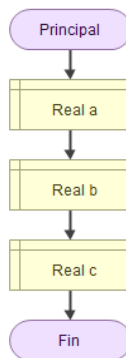
Aceptar Cancelar

Y doy clic en aceptar y la imagen queda de la siguiente manera:



Hago entonces lo mismo con los coeficientes b y c, y entonces el diagrama de flujo queda de la siguiente manera:

Asignatura	Datos del alumno	Fecha
Métodos Numéricos	Apellidos: De Mendoza Tovar	25-05-2025
	Nombre: Alejandro	



Ahora procedo con la **Resolución de la ecuación cuadrática ($\mathbb{R}$)**:

En este caso voy a denotar como valores para los coeficientes los siguientes (tener presente que se requiere sean números decimales):

$a = -7,5$
 $b = 17.9$
 $c = 21.3$

Y por ende la ecuación cuadrática entonces queda de la siguiente manera:

$$-7.5x^2 + 17.9x + 21.3 = 0$$

Ahora de mi parte es importante precisar que la fórmula cuadrática es:

$$\frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

Entonces ahora procedo a determinar el discriminante de la fórmula cuadrática que en este caso es $b^2 - 4ac$, y lo que me indica es que es el factor que discrimina entonces si voy a la ecuación cuadrática este me indica que es el argumento de la raíz la cual es cuadrada y lo que indica es que cualquier número que está dentro de la raíz debe ser positivo porque la raíz de un número negativo no existe y se incluye el 0 y esto es lo que debo garantizar.

Ahora de mi parte es preciso indicar que si el número que está dentro de la raíz es menor que 0 la raíz no existe y por lo tanto no tiene solución el sistema (tener presente que estamos trabajando dentro de los números reales). Entonces el discriminante puede ser mayor o igual que 0, o puede ser ≤ 0 , como se muestra a continuación:

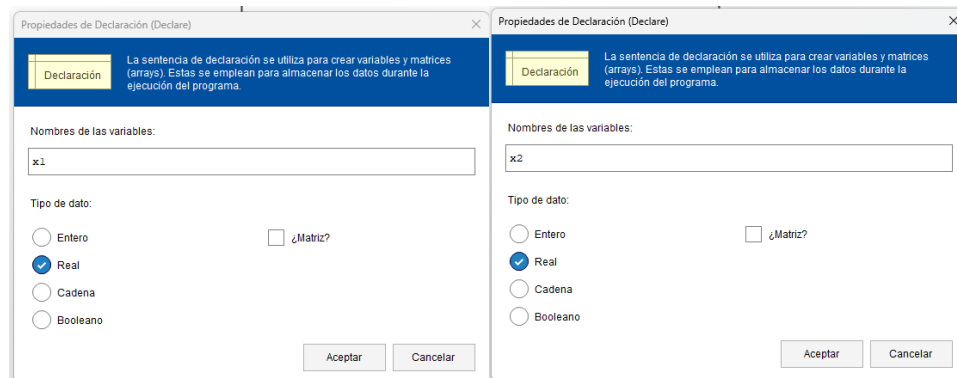
$$b^2 - 4ac \geq 0 \text{ ó } b^2 - 4ac > 0$$

Lo que se define es que el discriminante sea mayor o igual a 0, para la función de control, pero nunca menor que 0.

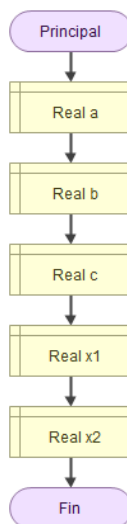
Ahora necesito declarar las variables de salida entonces:

1. Declaro una variable que va a ser una raíz que va a llamarse x1 y otra raíz que va a llamarse x2. Entonces el mismo proceso que ejecuté en puntos pasados y la imagen de la declaración queda de la siguiente manera:

Asignatura	Datos del alumno	Fecha
Métodos Numéricos	Apellidos: De Mendoza Tovar	25-05-2025
	Nombre: Alejandro	



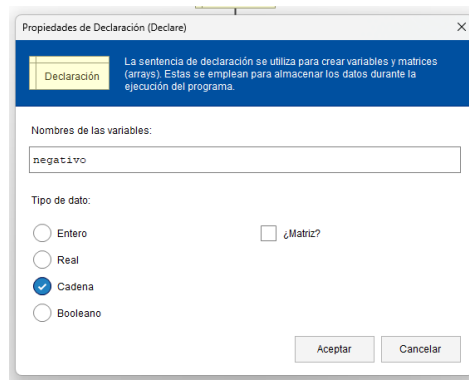
Y por ende el flujo con las variables de las dos raíces queda de la siguiente manera:



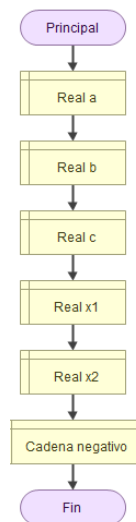
Entonces con base en esto tengo 3 coeficientes de entrada que son a, b y c y tengo dos coeficientes de salida que son dos raíces x1 y x2.

Ahora debo declarar otra variable en donde la función de control que en este caso es el discriminante pase por una de las dos vías, entonces si es ≥ 0 pues obtengo las dos raíces x1 y x2. Pero en el caso que sea < 0 debo declarar una variable de tipo texto en donde se le va a indicar al usuario que incorpora los coeficientes que el algoritmo se detiene porque el discriminante es negativo y por lo tanto la ecuación no tiene solución. Entonces declaro una variable de nombre "negativo" y va a ser una variable de tipo cadena, a continuación, la imagen:

Asignatura	Datos del alumno	Fecha
Métodos Numéricos	Apellidos: De Mendoza Tovar	25-05-2025
	Nombre: Alejandro	



Por ende, el flujo me queda entonces:

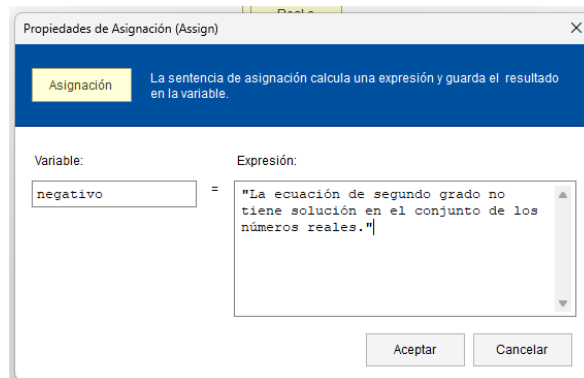


Pero no he terminado con esta variable, ahora a esta variable le voy a asignar un mensaje que va a ser: "La ecuación de segundo grado no tiene solución en el conjunto de los números reales.", y para esto doy clic en la línea, selecciono asignación

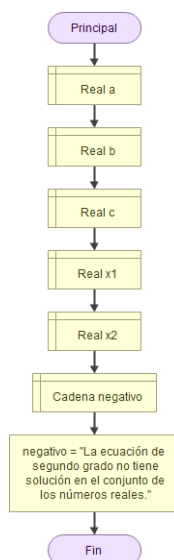


Luego doble clic en asignación, en el campo de variable coloco "negativo" y en la expresión coloco el mensaje "La ecuación de segundo grado no tiene solución en el conjunto de los números reales.", a continuación, la respectiva imagen:

Asignatura	Datos del alumno	Fecha
Métodos Numéricos	Apellidos: De Mendoza Tovar	25-05-2025
	Nombre: Alejandro	



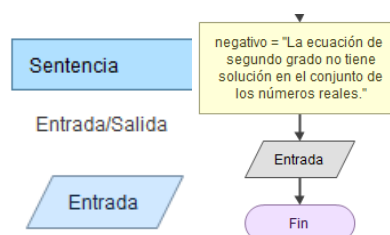
Doy aceptar y el flujo entonces me queda:



Listo ahora que hemos declarado todas las variables:

- Declaramos las variables a, b y c como variables reales.
- Declaramos x1 y x2 como variables reales de raíces
- Declaramos la variable de texto “negativo” con su respectivo mensaje.

Ahora voy a incorporar las variables de entrada, que ya como es claro ya las declare, entonces incorporo los coeficientes a través de una variable de entrada a, b y c, y para esto clic en la línea y en la parte de sentencia clic en entrada:



Y le doy como nombre de la variable, el coeficiente “a” y aplico lo mismo para b y c:

Asignatura	Datos del alumno	Fecha
Métodos Numéricos	Apellidos: De Mendoza Tovar	25-05-2025
	Nombre: Alejandro	

Propiedades de Entrada (Input)

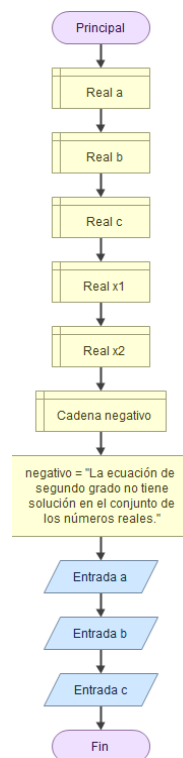
Entrada La sentencia Entrada (Input) lee un valor a través del teclado y lo almacena en una variable.

Nombre de la variable:

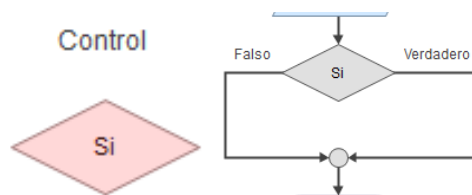
a

Aceptar Cancelar

Y el flujo queda entonces de la siguiente manera:



Ahora incorporo la función de control que es un SI o un IF, y lo que indica es que si pasa algo realiza una acción y si pasa lo contrario realiza la otra acción:

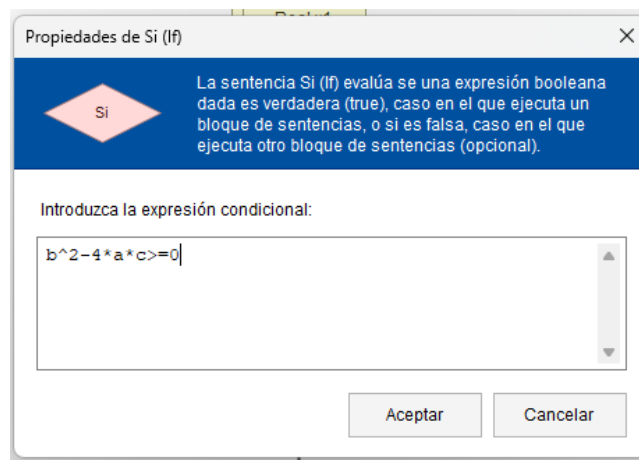


Entonces ahora establezco que se tiene que cumplir, entonces, lo que se debe cumplir es que:

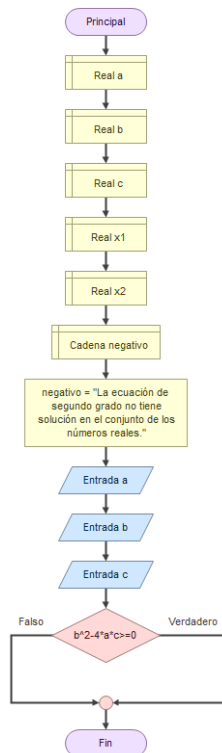
$$b^2 - 4ac \geq 0$$

A continuación, la imagen:

Asignatura	Datos del alumno	Fecha
Métodos Numéricos	Apellidos: De Mendoza Tovar	25-05-2025
	Nombre: Alejandro	



Y el flujo queda de la siguiente manera:



Ahora si es falso debe ejecutar una salida que indica la variable negativa que denote como “negativo”.

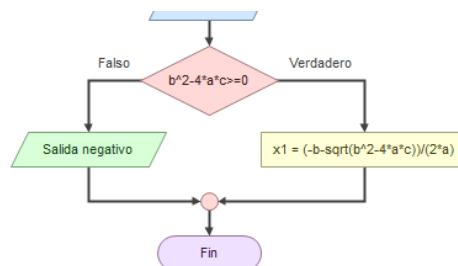


Asignatura	Datos del alumno	Fecha
Métodos Numéricos	Apellidos: De Mendoza Tovar	25-05-2025
	Nombre: Alejandro	

Por otro lado, si es verdadero ejecuta una salida que determina el cálculo de la raíz y para esto tenemos una **asignación** donde la variable x1 es igual a la fórmula cuadrática pero únicamente agregando el sino menos antes de la raíz:

$$\frac{-b - \sqrt{b^2 - 4ac}}{2a}$$

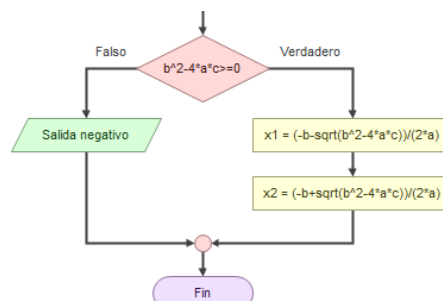
A continuación, la imagen:



Y ahora para la otra variable de salida x2 el resultado es la función cuadrática únicamente agregando el sino más antes de la raíz:

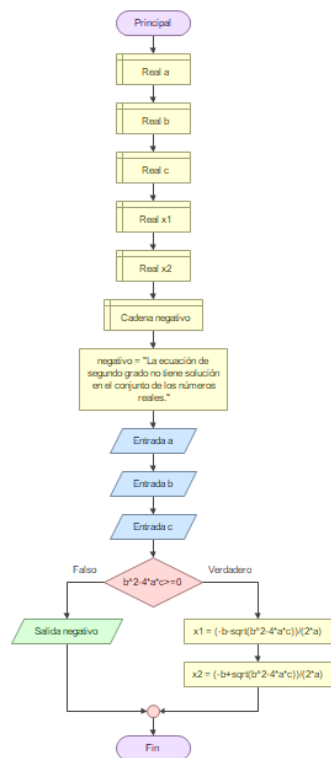
$$\frac{-b + \sqrt{b^2 - 4ac}}{2a}$$

A continuación, la imagen:

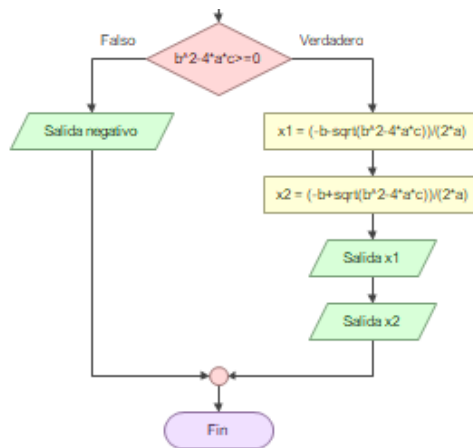


Ahora entonces el flujo completo queda de la siguiente manera:

Asignatura	Datos del alumno	Fecha
Métodos Numéricos	Apellidos: De Mendoza Tovar	25-05-2025
	Nombre: Alejandro	

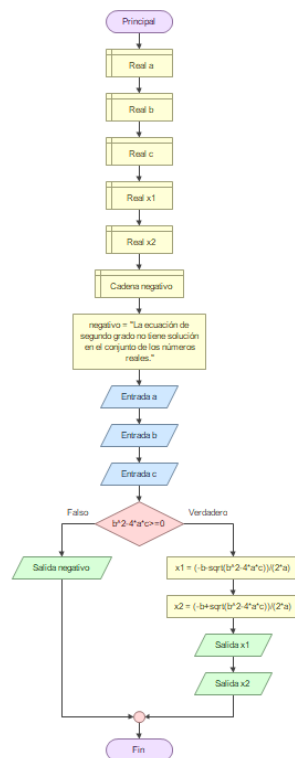


Finalmente agrego el resultado como una salida que es x1 y una salida que es x2



Y finalmente el flujo completo queda de la siguiente manera:

Asignatura	Datos del alumno	Fecha
Métodos Numéricos	Apellidos: De Mendoza Tovar	25-05-2025
	Nombre: Alejandro	



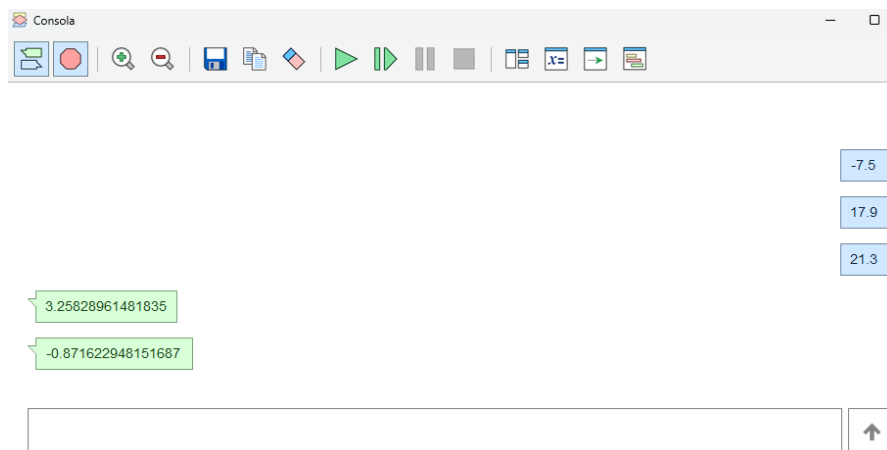
Ahora en la consola ingreso los coeficientes de a, b y c:

a= -7,5
b= 17.9
c = 21.3

Y el resultado que me muestra en la consola de las raíces es:

- X1 = 3.25828961481835
- X2 = -0.871622948151687

A continuación, la imagen respectiva de la consola:



Verificación con DESMOS

Asignatura	Datos del alumno	Fecha
Métodos Numéricos	Apellidos: De Mendoza Tovar	25-05-2025
	Nombre: Alejandro	

Método de verificación escrito:

Ahora procedo a comprobar los errores y para esto entonces utilizo otro método diferente al que se está utilizando en este proceso y para esto voy a “Desmos” a verificar el resultado entonces:

$a = -7.5$
 $b = 17.9$
 $c = 21.3$
 $x_1 = \frac{-b - \sqrt{b^2 - 4ac}}{2a} = 3.258289615$
 $x_2 = \frac{-b + \sqrt{b^2 - 4ac}}{2a} = -0.8716229482$

Entonces con esto se comprueba la validez del proceso realizado, ya que los resultados son los mismos.

Método de verificación gráfico:

Ahora procedí a mostrar la función gráficamente para dar un mayor entendimiento al desarrollo:



Entonces como se puede denotar:

- Es una parábola, porque es una función cuadrática.

Asignatura	Datos del alumno	Fecha
Métodos Numéricos	Apellidos: De Mendoza Tovar	25-05-2025
	Nombre: Alejandro	

- El coeficiente principal es -7.5, es decir, negativo. Por lo tanto, la parábola abre hacia abajo.
- El vértice se encuentra en:

$$X_v = -\frac{b}{2a} = -\frac{17.9}{2 * (-7.5)} \approx 1.193$$

- Entonces al sustituir el valor en la función:

$$f(1.193) \approx -7.5(1.193)^2 + 17.9(1.193) + 21.3 \approx 31.0$$

- Por lo tanto, el vértice es aproximadamente: (1.193, 31.0)
- Ahora frente a las intersecciones con el eje y, donde $x=0$, la función toma el valor del término independiente, entonces $f(0) = 21.3$ y por ende la parábola corta el eje Y en el punto (0, 21.3).
- Ahora, en cuanto a la solución de las raíces, la parábola corta el eje X en los puntos donde $f(x) = 0$, es decir, las soluciones de la ecuación cuadrática. Y por ende las raíces son:

$$x_1 \approx -0.8716, \quad x_2 \approx 3.2583$$

- Y entonces estos son los puntos donde la curva toca o cruza el eje X.
- Por último, a continuación, muestro una tabla que ayuda a visualizar completamente la función cuadrática:

Elemento	Valor aprox.
Concavidad	Hacia abajo
Vértice	(1.193, 31.0)
Intersección Y	(0, 21.3)
Raíces (X)	-0.8716 y 3.2583

Verificación con Google Colab

Ahora procedo a verificar el algoritmo y para esto voy a la ventana del visor del código fuente, cuyo lenguaje está en Python:

```

0  negativo = "La ecuación de segundo grado no tiene solución en el conjunto de los números reales."
1  a = float(input())
2  b = float(input())
3  c = float(input())
4  if b ** 2 - 4 * a * c >= 0:
5      x1 = -b - sqrt(b ** 2 - 4 * a * c) / 2 * a
6      x2 = -b + sqrt(b ** 2 - 4 * a * c) / 2 * a
7      print(x1)
8      print(x2)
9  else:
10     print(negativo)

```

Y lo que hago es copio el código en el portapapeles, luego voy a Google Colab y pego el código. A continuación, la imagen respectiva:

Asignatura	Datos del alumno	Fecha
Métodos Numéricos	Apellidos: De Mendoza Tovar	25-05-2025
	Nombre: Alejandro	

```

negativo = "La ecuación de segundo grado no tiene solución en el conjunto de los números reales."
a = float(input())
b = float(input())
c = float(input())
if b ** 2 - 4 * a * c >= 0:
    x1 = (-b - sqrt(b ** 2 - 4 * a * c)) / 2 * a
    x2 = (-b + sqrt(b ** 2 - 4 * a * c)) / 2 * a
    print(x1)
    print(x2)
else:
    print(negativo)

```

Ahora varias consideraciones que hay que incluir en el código:

- El código no trae la importación de la librería “math” por lo que es necesario importarla para poder ejecutar el código.
- De la librería “math” importo la función de raíz denotada con “sqrt”
- El código cuando lo copio y lo pego no trae los paréntesis en x1 y x2 (como se puede apreciar en el visor de código fuente de Flowgorithm por lo que, si no los coloco, el resultado trae error o un valor inexacto, es necesario colocarlos tanto para el divisor como para el numerador.
- Se procedió a dar instrucción de colocar valores de a, b y c para dar una mayor funcionalidad en la ejecución.
- De mi parte se procedió a cambiar en adición el print de X1 y de X2 para que tengan una mejor visualización.

A continuación, el código completo actualizado con los cambios:

```

import math
from math import sqrt
negativo = "La ecuación de segundo grado no tiene solución en el conjunto de los números reales."
a = float(input("Ingrese valor de a: "))
b = float(input("Ingrese valor de b: "))
c = float(input("Ingrese valor de c: "))
if b ** 2 - 4 * a * c >= 0:
    x1 = (-b - sqrt(b ** 2 - 4 * a * c)) / (2 * a)
    x2 = (-b + sqrt(b ** 2 - 4 * a * c)) / (2 * a)
    print("x1: ",x1)
    print("x2: ",x2)
else:
    print(negativo)

```

Y por ende al ejecutarlo con los coeficientes de a, b y c:

a= -7,5

Asignatura	Datos del alumno	Fecha
Métodos Numéricos	Apellidos: De Mendoza Tovar	25-05-2025
	Nombre: Alejandro	

b= 17.9
c = 21.3

El resultado nos da el resultado correcto:

- $X_1 = 3.258289614818354$
- $X_2 = -0.8716229481516876$

A continuación, la imagen de la plataforma de Google Colab con el resultado en la consola:

```

import math
from math import sqrt
negativo = "La ecuación de segundo grado no tiene solución en el conjunto de los números reales."
a = float(input("Ingrese valor de a: "))
b = float(input("Ingrese valor de b: "))
c = float(input("Ingrese valor de c: "))
if b ** 2 - 4 * a * c >= 0:
    x1 = (-b - sqrt(b ** 2 - 4 * a * c)) / (2 * a)
    x2 = (-b + sqrt(b ** 2 - 4 * a * c)) / (2 * a)
    print("x1: ",x1)
    print("x2: ",x2)
else:
    print(negativo)

```

Ingrese valor de a: -7.5
 Ingrese valor de b: 17.9
 Ingrese valor de c: 21.3
 x1: 3.258289614818354
 x2: -0.8716229481516876

Y con esto procedo a finalizar el desarrollo por el método tradicional.

2. Buscar una de las raíces de la ecuación de segundo grado usando el método de la Secante.

En este desarrollo de la segunda parte del laboratorio presento un análisis detallado del código implementado en Flowgorithm, que como es de claridad es un entorno de programación basado en diagramas de flujo, que utiliza el método de la secante para aproximar las raíces de una ecuación cuadrática de la forma $ax^2 + bx + c$.

Ahora para esta parte del laboratorio diseñe un programa para buscar **DOS raíces aproximadas utilizando dos intervalos iniciales distintos: uno en un rango negativo y otro en un rango positivo** (tener presente que solo se pide hallar una de las raíces).

Ahora iniciando este desarrollo lo primero considero respetado profesor es indicar que el método de la secante es una técnica numérica iterativa que se deriva del método de Newton-Raphson, pero en lugar de requerir la derivada de la función, utiliza la recta secante que pasa por dos puntos para estimar la siguiente aproximación a la raíz.

Es importante adicionalmente indicar que este enfoque es particularmente útil cuando la derivada no es fácil de calcular o cuando se desea una implementación más simple.

En este caso en el desarrollo y al analizar el mismo se puede denotar que el código se divide en tres funciones principales: la función **Principal**, que actúa como el controlador del programa; la función **evaluarFx**, que calcula el valor de la ecuación cuadrática para un valor

Asignatura	Datos del alumno	Fecha
Métodos Numéricos	Apellidos: De Mendoza Tovar	25-05-2025
	Nombre: Alejandro	

dado de x ; y la función **secante**, que implementa el algoritmo iterativo para encontrar las raíces.

A continuación, detallo cada componente con un enfoque específico, incluyendo el propósito de cada paso, las variables involucradas y el comportamiento esperado con los valores específicos proporcionados.

a. Valores de Entrada (Proporcionados en la Tabla)

La tabla adjunta especifica los valores de entrada que definen la ecuación y los parámetros del método de la secante. Estos son:

Variable	Valor	Comentario
a	-7.5	Coefficiente cuadrático
b	17.9	Coefficiente lineal
c	21.3	Constante
epsilon	0.00003	Criterio de parada (3 cienmilésimas)
maxIter	100	Número máximo de iteraciones (puede ser 100)
x0neg	-5	Inicio intervalo raíz negativa
x1neg	-1	Fin intervalo raíz negativa
x0pos	1	Inicio intervalo raíz positiva
x1pos	5	Fin intervalo raíz positiva

A continuación, procedo a dar una explicación más detallada sobre cada uno de ellos:

- $a = -7.5$: Coeficiente cuadrático de la ecuación. Indica que la parábola tiene una concavidad hacia abajo ya que $(a < 0)$, lo que sugiere que la función $f(x) = -7.5x^2 + 17.9x + 21.3$ tiene un máximo y potencialmente dos raíces reales si el discriminante es positivo.
- $b = 17.9$: Coeficiente lineal, que afecta la pendiente de la función lineal dentro de la ecuación cuadrática.
- $c = 21.3$: Término constante, que representa el valor de $f(x)$ cuando $x = 0$.
- $\epsilon = 0.00003$: Criterio de parada basado en la precisión. Este valor indica que el método considerará que ha encontrado una raíz cuando el valor absoluto de $f(x)$ sea menor a 0.00003 (es decir, una precisión de tres décimas de milésima). Este parámetro es crucial para determinar cuán cerca debe estar la aproximación del cero real de la función y como se indica y se instruye en clase es menor a 10 milésimas ya que son 3 cienmilésimas.
- $\text{maxIter} = 100$: Número máximo de iteraciones permitido. Este límite evita que el programa entre en un bucle infinito en caso de que el método no converja o converja muy lentamente.
- $x0neg = -5$: Punto inicial del intervalo negativo para buscar la primera raíz.
- $x1neg = -1$: Segundo punto inicial del intervalo negativo.
- $x0pos = 1$: Punto inicial del intervalo positivo para buscar la segunda raíz.
- $x1pos = 5$: Segundo punto inicial del intervalo positivo.

Estos valores definen la ecuación $-7.5x^2 + 17.9x + 21.3 = 0$ y establecen las condiciones iniciales para el método de la secante.

Ahora y de mi parte antes de proceder, considero y según lo indicado en clase, es útil calcular el discriminante para verificar si la ecuación tiene raíces reales:

Asignatura	Datos del alumno	Fecha
Métodos Numéricos	Apellidos: De Mendoza Tovar	25-05-2025
	Nombre: Alejandro	

$$\begin{aligned}
 - \text{Discriminante} &= b^2 - 4ac = 17.9^2 - 4 * -7.5 * 21.3 \\
 &= 320.41 + 638.4 \\
 &= 958.81
 \end{aligned}$$

Ahora y dado que el determinante es > 0 , la ecuación tiene dos raíces reales distintas, lo que, valida el uso de dos intervalos para buscarlas, por ende, procedo con el desarrollo completo por el método de la secante:

b. Estructura Detallada del Código

Para la estructura de este código tal como indique anteriormente este se divide en tres funciones principales: la función **Principal**, que actúa como el controlador del programa; la función **evaluarFx**, que calcula el valor de la ecuación cuadrática para un valor dado de (x); y la función **secante**, que implementa el algoritmo iterativo para encontrar las raíces. A continuación, procedo a dar la explicación en detalle de cada una de ellas:

i. Función Principal

La función Principal en este desarrollo de mi parte es el núcleo del programa, es la encargada de inicializar variables, solicitar o asignar valores, invocar la función secante para calcular las raíces y mostrar los resultados. A continuación, su desarrollo:

Paso 1: Declaración e Inicialización de Variables

Lo primero que hice fue declarar las siguientes variables como reales:

- a
- b
- c
- x0
- x0neg
- x0pos
- x1
- x1neg
- x1pos
- epsilon
- raiz1
- raiz2

Estas variables son las que almacenan los coeficientes de la ecuación, los puntos iniciales para el método de la secante y las raíces aproximadas. Ahora declare las variables enteras que para este caso solo es una:

- Declaro **maxIter** como entero, que controla el número máximo de iteraciones.

Entonces procedo ahora a asignar valores iniciales predeterminados:

- x0neg = -5: Define el punto de partida inferior para el intervalo negativo.
- x0pos = 1: Define el punto de partida inferior para el intervalo positivo.
- x1neg = -1: Define el punto de partida superior para el intervalo negativo.
- x1pos = 5: Define el punto de partida superior para el intervalo positivo.

Asignatura	Datos del alumno	Fecha
Métodos Numéricos	Apellidos: De Mendoza Tovar	25-05-2025
	Nombre: Alejandro	

Entonces estas asignaciones iniciales las desarrollo ya que me sirven como valores por defecto, pero debo indicar que en dado caso **el programa permite al usuario sobrescribirlos mediante entrada manual.**

A continuación, la respectivas imagen del flujo:



Paso 2: Entrada de Datos

En este desarrollo el programa solicita al usuario los valores de las variables clave. Sin embargo, como lo he implementado en el primer desarrollo que fue por el método del desarrollo de la cuadrática tradicional, los empleo en este desarrollo nuevamente y son los siguientes, y todos como es de claridad en valores reales:

Asignatura	Datos del alumno	Fecha
Métodos Numéricos	Apellidos: De Mendoza Tovar	25-05-2025
	Nombre: Alejandro	

Variable	Valor	Comentario
a	-7.5	Coeficiente cuadrático
b	17.9	Coeficiente lineal
c	21.3	Constante
epsilon	0.00003	Criterio de parada (3 cienmilésimas)
maxIter	100	Número máximo de iteraciones (puede ser 100)
x0neg	-5	Inicio intervalo raíz negativa
x1neg	-1	Fin intervalo raíz negativa
x0pos	1	Inicio intervalo raíz positiva
x1pos	5	Fin intervalo raíz positiva

Ahora con este paso simulo la interacción con el usuario, pero en este análisis se toman los valores fijos de la tabla para consistencia con el primer desarrollo como lo indique anteriormente.

Paso 3: Cálculo de las Raíces

En este desarrollo el programa realiza dos invocaciones a la función secante para aproximar las dos raíces:

- Primera llamada: `raiz1 = secante(-7.5, 17.9, 21.3, -5, -1, 0.00003, 100)`

Utiliza el intervalo $([-5, -1])$ para buscar una raíz, probablemente la raíz negativa o cercana a cero, dependiendo de cómo evolucione la iteración.

- Segunda llamada: `raiz2 = secante(-7.5, 17.9, 21.3, 1, 5, 0.00003, 100)`

Utiliza el intervalo $([1, 5])$ para buscar la otra raíz, que debería estar en el rango positivo.

La función secante devuelve las aproximaciones de las raíces, que se almacenan en `raiz1` y `raiz2`.

Paso 4: Salida de Resultados

- Se muestra el resultado de la primera raíz: `***Raíz 1 (aprox.): " & raiz1**`
- Se muestra el resultado de la segunda raíz: `***Raíz 2 (aprox.): " & raiz2**`

Ahora ejecuto estas salidas ya que estas permiten al usuario ver las aproximaciones calculadas, junto con la información adicional de iteraciones que proporciona la función secante durante su ejecución.

Paso 5: Fin del Programa

El flujo termina en el bloque "Fin", indicando que se han completado todas las operaciones y se han mostrado los resultados.

ii. Función evaluarFx

Para esta etapa del desarrollo indico que en este caso la función `evaluarFx` es una subrutina auxiliar que calcula el valor de la ecuación cuadrática $f(x) = ax^2 + bx + c$ para un valor

Asignatura	Datos del alumno	Fecha
Métodos Numéricos	Apellidos: De Mendoza Tovar	25-05-2025
	Nombre: Alejandro	

específico de (x) . Es esencial para el método de la secante, ya que necesita evaluar la función en los puntos iniciales y en las aproximaciones sucesivas.

Parámetros

- a, b, c, x (todos reales): Representan los coeficientes de la ecuación y el valor de (x) en el que se evalúa la función.

Variables Locales

- evaluarFx (real): Variable que almacenará y retornará el resultado del cálculo.
- resultado (real): Variable intermedia que realiza el cálculo paso a paso.

Proceso Detallado

Cálculo de $(f(x))$:

- Se asigna: $\text{resultado} = a * x * x + b * x + c$
- Con los valores dados: $\text{resultado} = -7.5 * x * x + 17.9 * x + 21.3$
- Este paso multiplica el coeficiente cuadrático (a) por (x^2) , suma el término lineal (bx) y agrega la constante (c). La operación sigue el orden estándar de evaluación matemática, asegurando que la potencia se calcule primero.

Asignación del resultado:

- $\text{evaluarFx} = \text{resultado}$
- El valor calculado se transfiere a la variable de retorno.

Retorno:

- La función retorna evaluarFx, que es el valor numérico de $f(x)$ para el (x) ingresado.

Importancia Matemática:

Esta función es el corazón del análisis numérico, ya que proporciona los valores de $f(x)$ necesarios para determinar si un punto es una raíz (el valor de la función $f(x)$ es aproximadamente igual a cero) o para calcular la pendiente aproximada en el método de la secante. Por ejemplo:

- $(x = -5): (f(-5)) = -7.5 * (-5)^2 + 17.9 * (-5) + 21.3 = -7.5 * 25 - 89.5 + 21.3 = -187.5 - 89.5 + 21.3 = -255.7$
- $(x = 1): (f(1)) = -7.5 * 1^2 + 17.9 * 1 + 21.3 = -7.5 + 17.9 + 21.3 = 31.7$

Estos cálculos muestran cómo la función evalúa la ecuación en diferentes puntos, lo que es fundamental para el proceso iterativo.

iii. Función secante

La función secante implementa el algoritmo del método de la secante, un procedimiento iterativo que aproxima una raíz de $(f(x) = 0)$ utilizando dos puntos iniciales (x_0) y (x_1) . La idea detrás del método es trazar una recta (la secante) que conecte los puntos $((x_0, f(x_0)))$ y $((x_1, f(x_1)))$.

Asignatura	Datos del alumno	Fecha
Métodos Numéricos	Apellidos: De Mendoza Tovar	25-05-2025
	Nombre: Alejandro	

$f(x_1))$, y encontrar el punto donde esta recta cruza el eje (x) (es decir, donde $(f(x) = 0)$). Este punto se usa como la siguiente aproximación.

Parámetros:

- a, b, c (reales): Coeficientes de la ecuación cuadrática.
- x_0 , x_1 (reales): Puntos iniciales del intervalo.
- epsilon (real): Precisión deseada para la raíz.
- maxIter (entero): Límite de iteraciones.

Variables Locales:

- secante (real): Almacena la raíz aproximada que se retornará.
- fx_0 , fx_1 (reales): Valores de la función ($f(x)$) evaluados en (x_0) y (x_1).
- xNuevo (real): Nueva aproximación calculada en cada iteración.
- iteracion (entero): Contador que lleva la cuenta de las iteraciones realizadas.
- terminate (booleano): Bandera que indica si el método debe detenerse.

Proceso Detallado:

Inicialización

- iteracion = 0: Comienza el conteo de iteraciones desde cero.
- terminate = False: Inicialmente, no hay razón para detener el bucle.

Bucle Principal (Mientras iteracion < maxIter AND NOT terminate)

Este bucle es el corazón del método de la secante y se ejecuta hasta que se cumpla una condición de parada. A continuación, describo los pasos dentro del bucle de manera específica:

Evaluación de la función

- fx_0 = evaluar $Fx(a, b, c, x_0)$: Calcula el valor de $f(x_0)$ usando la función auxiliar.
- fx_1 = evaluar $Fx(a, b, c, x_1)$: Calcula el valor de $f(x_1)$.

Para el intervalo [-5, -1]:

$$\begin{aligned} fx_0 &= f(-5) = -255.7 \\ fx_1 &= f(-1) = -4.1 \end{aligned}$$

Para el intervalo [1, 5]:

$$\begin{aligned} fx_0 &= f(1) = 31.7 \\ fx_1 &= f(5) = -76.7 \end{aligned}$$

Estos valores iniciales determinan la dirección y magnitud de la pendiente de la secante.

Verificación de Precisión

Para este proceso entonces:

Asignatura	Datos del alumno	Fecha
Métodos Numéricos	Apellidos: De Mendoza Tovar	25-05-2025
	Nombre: Alejandro	

- Se evalúa la condición $\text{abs}(f(x_1)) < \text{epsilon}$, es decir, $|f(x_1)| < 0.00003$.
- Si se cumple, significa que (x_1) es una aproximación suficientemente cercana a una raíz donde $f(x) \approx 0$ o el valor de la función $f(x)$ es aproximadamente igual a cero.

Acciones:

- Imprime: "Raíz: " & x_1 & ", Iteraciones: " & iteracion
- Asigna: secante = x_1
- Asigna: terminate = true

Esto detiene el bucle porque se ha alcanzado la precisión deseada.

Verificación de Convergencia

- Se verifica si $f(x_1) - f(x_0) = 0$.

Si el denominador de la fórmula de la secante $f(x_1) - f(x_0)$ es cero, el método no puede continuar porque la división por cero es indefinida, lo que indica que el método no convergerá en este intervalo.

Acciones:

- Imprime: "Error: el método no converge."
- Asigna: secante = 999999999 (un valor grande que actúa como indicador de error).
- Asigna: terminate = true

Esto detiene el bucle y notifica al usuario del fallo.

Cálculo de la Nueva Aproximación

Si ninguna de las condiciones anteriores se cumple, se calcula el nuevo punto usando la fórmula del método de la secante:

$$x_{\text{Nuevo}} = x_1 - f(x_1) * \frac{x_1 - x_0}{(f(x_1) - f(x_0))}$$

Esta fórmula deriva de la intersección de la recta secante con el eje (x) . Geométricamente, representa el punto donde la recta que pasa por $(x_0, f(x_0))$ y $(x_1, f(x_1))$ cruza $f(x) = 0$.

Actualización de variables

- $x_0 = x_1$: El valor anterior de (x_1) se convierte en el nuevo (x_0) .
- $x_1 = x_{\text{Nuevo}}$: El nuevo punto calculado reemplaza a (x_1) .
- Incremento: iteracion = iteracion + 1
- Salida: Imprime "Iteraciones realizadas: " & iteracion para informar al usuario del progreso.

Salida Final del Bucle

Una vez que el bucle termina (por alcanzar maxIter o por terminate = true), se imprime:

- "Raíz: " & x_1 & ", Iteraciones: " & iteracion
- Se asigna: secante = x_1 , asegurando que el último valor de (x_1) sea la raíz aproximada.

Asignatura	Datos del alumno	Fecha
Métodos Numéricos	Apellidos: De Mendoza Tovar	25-05-2025
	Nombre: Alejandro	

Retorno:

- La función retorna secante, que es la aproximación final de la raíz.

Análisis Matemático del Método de la Secante:

La fórmula $(x_{\text{Nuevo}} = x_1 - f(x_1) * (x_1 - x_0) / (f(x_1) - f(x_0)))$ se deriva de la ecuación de la recta secante. La pendiente de la secante es $(f(x_1) - f(x_0)) / (x_1 - x_0)$, y el punto donde esta recta cruza el eje (x) se calcula resolviendo $f(x_1) + m(x - x_1) = 0$, donde (m) es la pendiente. Simplificando, se obtiene la expresión utilizada. Este método tiene una convergencia super lineal (mejor que lineal, pero no cuadrática como Newton-Raphson), siempre que los puntos iniciales estén lo suficientemente cerca de la raíz y $f(x)$ sea continua y diferenciable en el intervalo.

Aplicación del Método de la Secante con los Valores Dados

- Ecuación: $-7.5x^2 + 17.9x + 21.3 = 0$
- Intervalo para raíz1: [-5, -1]
- Intervalo para raíz2: [1, 5]
- Precisión: El método se detiene cuando $|f(x)| < 0.00003$
- Máximo de iteraciones: 100

Comportamiento Esperado

Raíz 1 (intervalo [-5, -1]):

- Los valores iniciales ($x_0 = -5$) y ($x_1 = -1$) producen $f(-5) = -255.7$ y $f(-1) = -4.1$, indicando que la función cambia de signo en algún punto intermedio (por el teorema del valor intermedio, hay al menos una raíz en este intervalo si $f(x)$ es continua, lo cual es cierto para una cuadrática).
- El método iterará, ajustando (x_0) y (x_1), hasta que $|f(x_1)| < 0.00003$ o se agoten las 100 iteraciones.
- Dado que el discriminante es positivo, se espera que el método converja a una raíz (probablemente negativa o cercana a cero).

Raíz 2 (intervalo [1, 5]):

- Los valores iniciales ($x_0 = 1$) y ($x_1 = 5$) producen $f(1) = 31.7$ y $f(5) = -76.7$, mostrando un cambio de signo, lo que sugiere la presencia de una raíz en este intervalo.
- El método iterará de manera similar, convergiendo hacia la otra raíz (probablemente positiva).

Conclusión

Este programa implementa de manera efectiva el método de la secante para aproximar las raíces de la ecuación $-7.5x^2 + 17.9x + 21.3 = 0$, utilizando los intervalos [-5, -1] y [1, 5]. La función evaluarFx proporciona los valores necesarios para las iteraciones, mientras que secante aplica el algoritmo iterativo con un control de precisión (epsilon = 0.00003) y un límite de iteraciones (maxIter = 100). El resultado final muestra las raíces aproximadas junto con el número de iteraciones, ofreciendo una herramienta útil para resolver ecuaciones cuadráticas numéricamente. Para una aplicación práctica, sería ideal validar los intervalos iniciales y ajustar (epsilon) según la precisión requerida.

Ingreso de valores y pruebas

Ahora en la consola ingreso los coeficientes de a, b y c:

a= -7,5

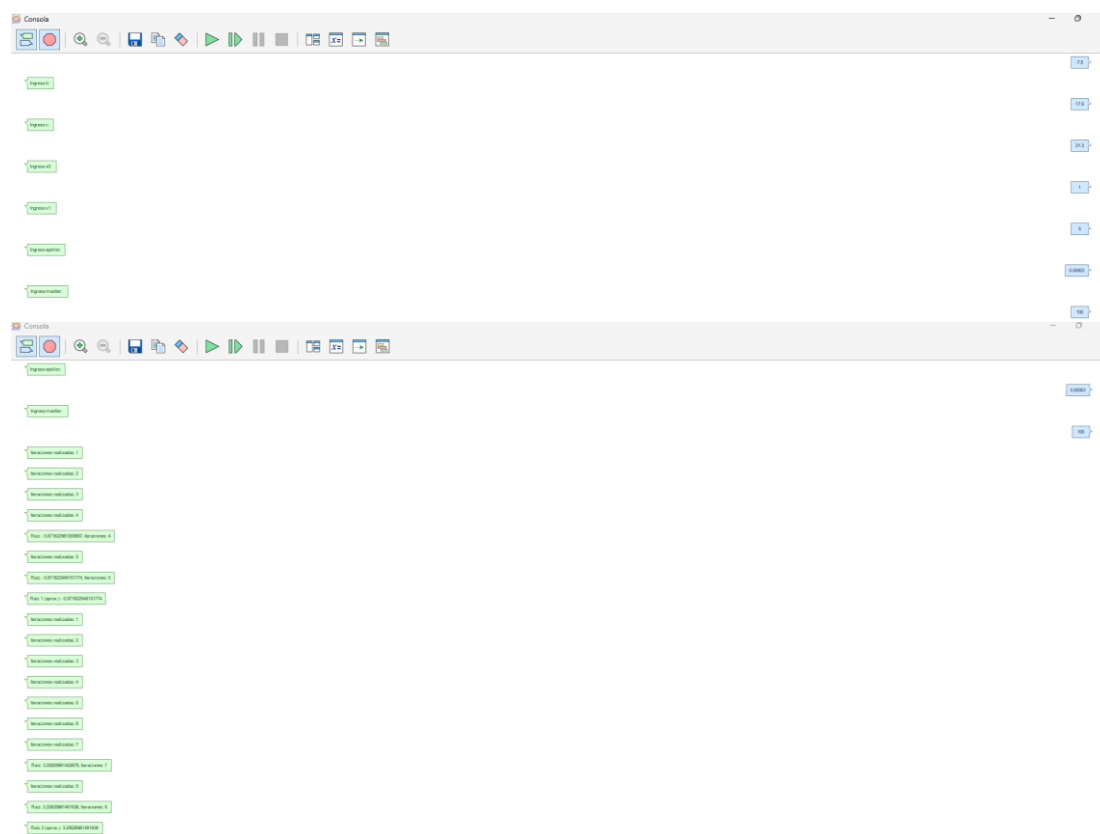
Asignatura	Datos del alumno	Fecha
Métodos Numéricos	Apellidos: De Mendoza Tovar	25-05-2025
	Nombre: Alejandro	

b= 17.9
c = 21.3

Y el resultado que me muestra en la consola de las raíces es:

- $X1 = 3.25828961481835$
- $X2 = -0.871622948151687$

A continuación, la imagen respectiva de la consola en Flowgorithm:



Verificación con DESMOS

Método de verificación escrito:

Ahora procedo a comprobar los errores y para esto entonces utilizo otro método diferente al que se está utilizando en este proceso y para esto voy a "Desmos" a verificar el resultado entonces:

Asignatura	Datos del alumno	Fecha
Métodos Numéricos	Apellidos: De Mendoza Tovar	25-05-2025
	Nombre: Alejandro	

$a = -7.5$	$= -7.5$
$b = 17.9$	$= 17.9$
$c = 21.3$	$= 21.3$
$x_1 = \frac{-b - \sqrt{b^2 - 4ac}}{2a}$	$= 3.258289615$
$x_2 = \frac{-b + \sqrt{b^2 - 4ac}}{2a}$	$= -0.8716229482$

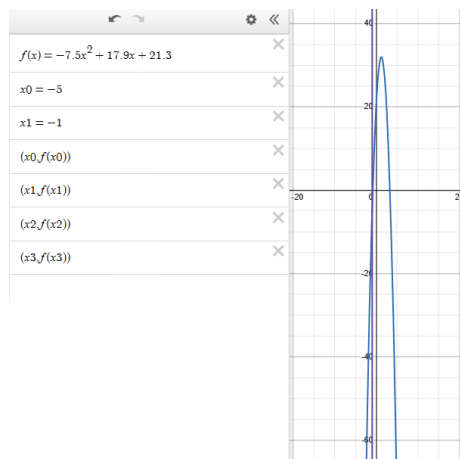
ppal abc fnc RAD GRAD
borrar todo

a^2	a^b	$ a $	7	8	9	$\div$	%	$\frac{a}{b}$
$\sqrt{}$	$\sqrt[n]{}$	π	4	5	6	$\times$	$\leftarrow$	$\rightarrow$
sin	cos	tan	1	2	3	$-$	$\leftarrow$	$\rightarrow$
(	)	,	0	.	ans	+	$\rightarrow$	

Entonces con esto se comprueba la validez del proceso realizado, ya que los resultados son los mismos.

Método de verificación gráfico:

Ahora procedí a mostrar la función gráficamente para dar un mayor entendimiento al desarrollo:



Ahora se remite imagen de la gráfica con $x_0 = -5$ y $x_1 = -1$, ya que se procedió a revisar cómo cambian las aproximaciones en las iteraciones.

Y para finalizar como se puede observar los resultados son los mismos por lo que el desarrollo esta correcto y con esto termino el desarrollo del método de la secante.

Verificación con Google Colab

Ahora procedo a verificar el algoritmo y para esto voy a la ventana del visor del código en Flowgorithm, cuyo lenguaje está en Python:

Asignatura	Datos del alumno	Fecha
Métodos Numéricos	Apellidos: De Mendoza Tovar	25-05-2025
	Nombre: Alejandro	

```

Visor de código fuente
Python

def evaluarFx(a, b, c, x):
    resultado = a * x ** 2 + b * x + c
    evaluarFx = resultado
    return evaluarFx

def secante(a, b, c, x0, x1, epsilon, maxiter):
    iteration = 0
    terminate = False
    while iteration < maxiter and not terminate:
        fx0 = evaluarFx(a, b, c, x0)
        fx1 = evaluarFx(a, b, c, x1)
        if abs(fx1) < epsilon:
            print("Raíz: " + str(x1) + ", Iteraciones: " + str(iteration))
            terminate = True
        if fx0 == fx1:
            print("Error: el método no converge.")
            secante = 999999999
            terminate = True
        else:
            xNuevo = x1 - fx1 * (x1 - x0) / (fx1 - fx0)
            x0 = x1
            x1 = xNuevo
            iteration = iteration + 1
            print("Iteraciones realizadas: " + str(iteration))
        print("Raíz: " + str(x1) + ", Iteraciones: " + str(iteration))
        secante = x1
    return secante

# Main
x0neg = -5
x0pos = 1
x1neg = -1
x1pos = 1
epsilon = 1e-6
maxiter = 1000

a = float(input())
b = float(input())
c = float(input())
x0 = float(input())
x1 = float(input())
epsilon = float(input())
maxiter = int(input())
raiz1 = secante(a, b, c, x0neg, x1neg, epsilon, maxiter)
raiz2 = secante(a, b, c, x0pos, x1pos, epsilon, maxiter)
print("Raíz 1 (aprox.): " + str(raiz1))
print("Raíz 2 (aprox.): " + str(raiz2))

```

Y lo que hago es copio el código en el portapapeles, luego voy a Google Colab y pego el código. A continuación, la imagen respectiva:

```

import math
from math import sqrt
def evaluarFx(a, b, c, x):
    resultado = a * x ** 2 + b * x + c
    evaluarFx = resultado
    return evaluarFx

def secante(a, b, c, x0, x1, epsilon, maxiter):
    iteration = 0
    terminate = False
    while iteration < maxiter and not terminate:
        fx0 = evaluarFx(a, b, c, x0)
        fx1 = evaluarFx(a, b, c, x1)
        if abs(fx1) < epsilon:
            print("Raíz: " + str(x1) + ", Iteraciones: " + str(iteration))
            terminate = True
        if fx0 == fx1:
            print("Error: el método no converge.")
            secante = 999999999
            terminate = True
        else:
            xNuevo = x1 - fx1 * (x1 - x0) / (fx1 - fx0)
            x0 = x1
            x1 = xNuevo
            iteration = iteration + 1
            print("Iteraciones realizadas: " + str(iteration))
        print("Raíz: " + str(x1) + ", Iteraciones: " + str(iteration))
        secante = x1
    return secante

# Main
x0neg = -5
x0pos = 1
x1neg = -1
x1pos = 1
epsilon = 1e-6
maxiter = 1000

a = float(input())
b = float(input())
c = float(input())
x0 = float(input())
x1 = float(input())
epsilon = float(input())
maxiter = int(input())
raiz1 = secante(a, b, c, x0neg, x1neg, epsilon, maxiter)
raiz2 = secante(a, b, c, x0pos, x1pos, epsilon, maxiter)
print("Raíz 1 (aprox.): " + str(raiz1))
print("Raíz 2 (aprox.): " + str(raiz2))

```

El resultado como podemos denotar nos da nuevamente el resultado correcto:

- $X1 = 3.258289614818354$
- $X2 = -0.8716229481516876$

Asignatura	Datos del alumno	Fecha
Métodos Numéricos	Apellidos: De Mendoza Tovar	25-05-2025
	Nombre: Alejandro	

Y con esto procedo a finalizar el desarrollo por el método de la secante.

3. Diagrama de flujo con su respectiva sustentación matemática y código fuente generado para el programa que encuentra las raíces reales de la ecuación $ax^2+bx+c=0$, junto con una captura de pantalla con la ejecución en el IDE elegido.

A continuación, brindo las respectivas imágenes del flujo por el método principal, y código fuente generado para el programa que encuentra las raíces reales de la ecuación $ax^2 + bx + c = 0$, junto con una captura de pantalla con la ejecución en los IDE elegidos.

Flujo, código en Python y salida de consola en Flowgorithm

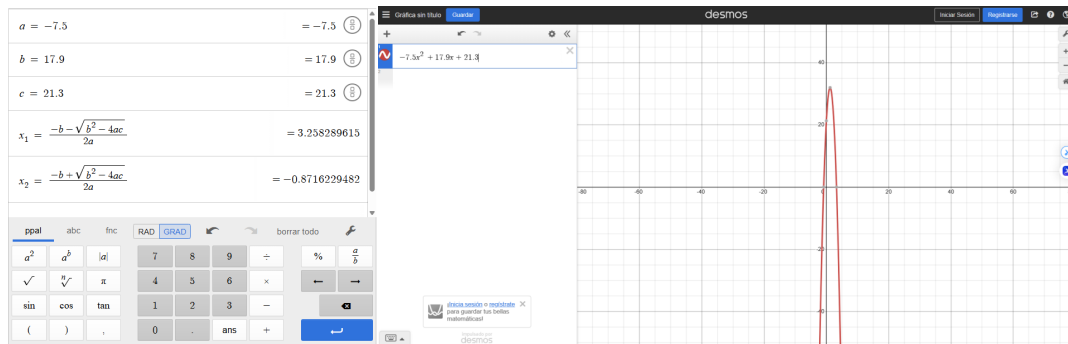
The screenshot shows the Flowgorithm IDE interface. On the left, a flowchart outlines the logic for solving a quadratic equation. It starts with 'Inicio', followed by input steps for 'a', 'b', and 'c'. A decision diamond checks if the discriminant $b^2 - 4ac \geq 0$. If true, it calculates two roots, x_1 and x_2 , and prints them. If false, it prints a message indicating no real solutions. The flowchart ends with 'Fin'. On the right, the corresponding Python code is displayed in a window titled 'Visor de código fuente'. The code uses `float(input())` for inputs, `if b**2 - 4*a*c >= 0:` for the discriminant check, and `print(x1)`, `print(x2)` for the roots. Below the code, a console window shows the execution results for inputs -7.5, 17.9, and 21.3, resulting in roots 3.258289614818354 and -0.871622948151687.

Código y salida de consola en Google Colab

The screenshot shows a Google Colab notebook. The code cell contains the same Python code as the Flowgorithm IDE. The output cell shows the execution results for inputs -7.5, 17.9, and 21.3, resulting in roots 3.258289614818354 and -0.871622948151687.

Código, salida y grafica de consola en DESMOS

Asignatura	Datos del alumno	Fecha
Métodos Numéricos	Apellidos: De Mendoza Tovar	25-05-2025
	Nombre: Alejandro	

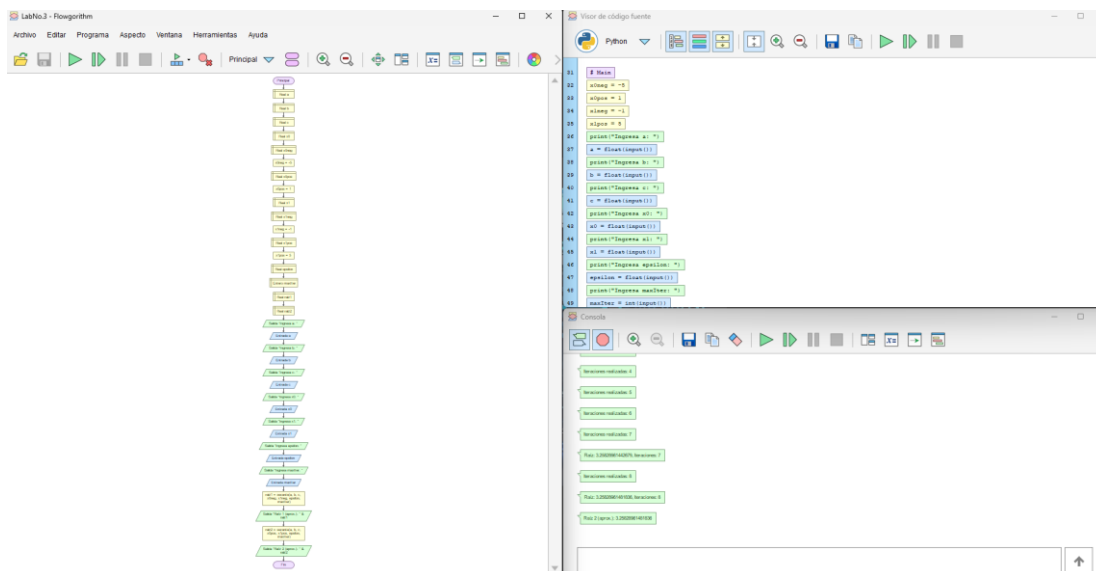


- Diagrama de flujo con su respectiva sustentación matemática y código fuente generado para la técnica de la secante que busca una de las raíces de la ecuación de segundo grado, junto con una captura de pantalla con la ejecución en el IDE elegido.

Para este punto como es de claridad aclaro que en la respuesta de la consola se pueden denotar las iteraciones realizadas. Además, este flujo se compone de 3 imágenes una es la de la función principal, otra la de la función evaluarFx y la otra es la de la función secante, a continuación, las imágenes respectivas:

Flujo, código en Python y salida de consola en Flowgorithm

Función Principal:



Función evaluarFx:

Asignatura	Datos del alumno	Fecha
Métodos Numéricos	Apellidos: De Mendoza Tovar	25-05-2025
	Nombre: Alejandro	

The image shows two side-by-side windows. The left window is 'LabNo.3 - Flowgorithm', displaying a flowchart for a function evaluation. The flowchart starts with a process box 'evaluarFx (Real a, Real b, Real c, Real x)', followed by a data box 'Real evaluarFx', then a data box 'Real resultado', a calculation box 'resultado = a * x^2 + b * x + c', another data box 'evaluarFx = resultado', and finally a return box 'Retorno Real evaluarFx'. The right window is a Python code editor showing the implementation of the Secant method. It defines a function 'evaluarFx(a, b, c, x)' and a 'secante' function. The 'secante' function takes parameters 'a, b, c, x0, x1, epsilon, maxIter' and implements the Secant method algorithm with a while loop for iterations, calculating the next point 'x2' using the formula $x_2 = x_1 - \frac{f(x_1)(x_1 - x_0)}{f(x_1) - f(x_0)}$. It prints the iteration details and returns the final result 'x2'.

Función Secante:

This image is similar to the one above but shows a more complex flowchart in the Flowgorithm window. The flowchart for the 'secante' function includes decision boxes for 'maxIter < 0', 'epsilon < 0', and 'abs(x2 - x1) < epsilon'. It also includes a 'break' box and a 'return x2' box. The Python code editor on the right shows the same implementation as before, but with a 'break' statement added to the while loop when the iteration limit is reached. The console output shows the results of the iterations, including the final value of 'x2' and the number of iterations.

Código y salida de consola en Google Colab (tener presente que las iteraciones se muestran en consola)

Asignatura	Datos del alumno	Fecha
Métodos Numéricos	Apellidos: De Mendoza Tovar	25-05-2025
	Nombre: Alejandro	

```

import math
from math import sqrt
from math import fabs
def evaluarFx(a, b, c, x):
    resultado = a * x ** 2 + b * x + c
    evaluarFx = resultado
    return evaluarFx

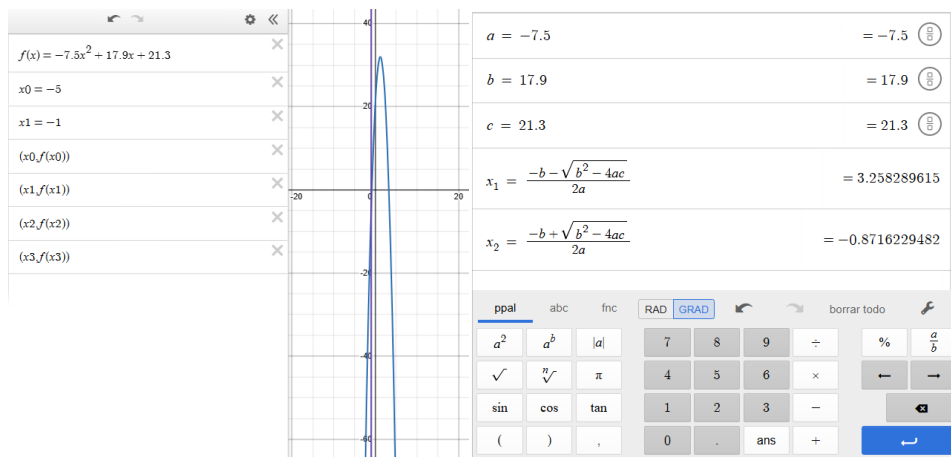
def secante(a, b, c, x0, x1, epsilon, maxiter):
    iteration = 0
    terminate = False
    while iteration < maxiter and not terminate:
        fx0 = evaluarFx(a, b, c, x0)
        fx1 = evaluarFx(a, b, c, x1)
        if fabs(fx0) < epsilon:
            print("Raiz: " + str(x0) + ", Iteraciones: " + str(iteration))
            secante = x0
            terminate = True
        if fx1 - fx0 == 0:
            print("Error: el método no converge.")
            secante = 999999999
            terminate = True
        else:
            xNuevo = x1 - fx1 * (x1 - x0) / (fx1 - fx0)
            x0 = x1
            x1 = xNuevo
            iteration = iteration + 1
            print("Iteraciones realizadas: " + str(iteration))
            print("Raiz: " + str(x0) + ", Iteraciones: " + str(iteration))
            secante = x1
    return secante

# Main
x0neg = -5
x0pos = 1
x1neg = -1
x1pos = 5
print("Ingresar a:")
a = float(input())
print("Ingresar b:")
b = float(input())
print("Ingresar c:")
c = float(input())
print("Ingresar x0:")
x0 = float(input())
print("Ingresar x1:")
x1 = float(input())
print("Ingresar epsilon:")
epsilon = float(input())
print("Ingresar maxiter:")
maxiter = int(input())
raiz1 = secante(a, b, c, x0neg, x1neg, epsilon, maxiter)
print("Raiz 1 (aprox.): " + str(raiz1))
raiz2 = secante(a, b, c, x0pos, x1pos, epsilon, maxiter)
print("Raiz 2 (aprox.): " + str(raiz2))

```

Código, salida y grafica de consola en DESMOS

Se remite imagen de la grafica con $x_0 = -5$ y $x_1 = -1$, ya que se procedió a revisar cómo cambian las aproximaciones en las iteraciones.



Y con esto finalizo el desarrollo de este punto.

CONCLUSIÓN

El desarrollo de este laboratorio fue una oportunidad valiosa para integrar y aplicar diversos conceptos relacionados con algoritmos, métodos numéricos y programación estructurada. Una de las herramientas clave fue el uso de diagramas de flujo, que me permitió visualizar de manera clara y ordenada la lógica del programa antes de implementarlo en código. Considero que una de las principales ventajas de esta herramienta es su capacidad para facilitar la

Asignatura	Datos del alumno	Fecha
Métodos Numéricos	Apellidos: De Mendoza Tovar	25-05-2025
	Nombre: Alejandro	

comprensión y detección temprana de errores lógicos. Sin embargo, también identifiqué como desventaja que, en algoritmos más complejos, los diagramas pueden volverse extensos y menos prácticos para representar cada detalle del flujo de control.

Asimismo, las pruebas de escritorio jugaron un papel fundamental en la validación del algoritmo antes de su ejecución en el entorno de programación. Este paso me permitió verificar que las variables fueran correctamente inicializadas, que las iteraciones se comportaran como se esperaba, y que los resultados numéricos fueran coherentes. Al detectar inconsistencias, pude realizar ajustes puntuales al código fuente, especialmente al trasladarlo de Flowgorithm a Python, adaptando la sintaxis, manejando condiciones de parada, y afinando los valores de tolerancia e iteraciones para mejorar la precisión del método de la secante.

Además, la comparación entre el método tradicional y el método de la secante me permitió comprender mejor las ventajas de los métodos numéricos cuando se requieren aproximaciones rápidas en contextos donde una solución exacta no es viable o no está disponible. Finalmente, el uso de herramientas gráficas como Desmos complementó el análisis al ofrecer una validación visual de los resultados obtenidos.

En conclusión, este laboratorio fortaleció no solo mis habilidades técnicas, sino también mi capacidad de análisis y de integración de herramientas computacionales en la solución de problemas matemáticos reales, destacando la importancia de planificar, validar y ajustar cada etapa del proceso de desarrollo.

Muchas gracias, profesor, por brindarme estos grandes conceptos que me darán apoyo en mi futuro profesional.

BIBLIOGRAFÍA

Respetado profesor Ing. Hernán Alonso a continuación, la bibliografía implementada:

- ✓ Tema 1. Principios fundamentales del conteo. Tema 2. Análisis numérico y de los errores. Tema 3. Aritmética del computador. Tema 4. Cálculo de raíces e interpolación. Tema 5. Algoritmos de resolución y técnicas de aceleración,
- ✓ Burden, R. L., & Faires, J. D. (2011). Análisis Numérico (9.ª ed.). Cengage Learning.
- ✓ Referencia clave para la explicación y justificación del método de la secante.
- ✓ Sullivan, M. (2013). Álgebra y Trigonometría (9.ª ed.). Pearson.
- ✓ Para fundamentos sobre la resolución de ecuaciones cuadráticas
- ✓ Clases virtuales con el profesor Ing. Deivis Eduard Ramírez Martínez.
- ✓ Material de estudio del aula.